

av-stack-defense

Cross-Layer Signal Fusion

Multi-Agent Collaboration Report

Design, division of labor, capabilities, and handoffs for fusing five independent attack detectors into a single Joint Threat Score that catches coordinated, sub-threshold attacks across the autonomous-vehicle stack.

Project: Cross-Layer Autonomous-Vehicle Defense Framework

Feature: Cross-Layer Signal Fusion (Joint Threat Score)

Design authority: Claude Code (Architect & Algorithm Design)

Collaborators: OpenAI Codex (implementation), Antigravity (visualization)

Primary spec artifact: FUSION_SPEC.md

Status: Step 1 (fusion architecture) complete and handed off

1. Objective

Each detector layer already raises its own local alarm when its metric crosses a hard threshold. Those alarms are blind to one another. A capable adversary can hold every individual layer just under its threshold while nudging several at once, and no single detector fires.

Cross-layer fusion closes that gap. It maps every layer's raw metric onto one common anomaly scale, combines them into a Joint Threat Score τ_{joint} , and raises a global alarm when several layers are jointly elevated even if none crosses its own threshold.

2. Division of Labor

Agent	Specialty role	Target
Claude Code	Architect & Algorithm Design	Design the fusion math and specification
OpenAI Codex	Python Developer	Implement the fusion inside <code>harness.py</code>
Antigravity	UI-UX Designer & Integrator	Render the live Cross-Layer Threat Alert panel in the dashboard

This report's authority. Claude Code owns Step 1. The math, the data contract, and the reference implementation in this document are the source of truth that Steps 2 and 3 build against.

3. The Three-Step Plan

STEP 1 Fusion Architecture — Claude Code

Design a Joint Threat Score $\tau_{\text{joint}} = \sum w_i \cdot \phi_i(\text{metric}_i)$, where ϕ_i is a normalized anomaly value per layer. The score triggers a global alarm when multiple layers show minor anomalies at the same time, even when no single layer crosses its own threshold. Deliverable: `FUSION_SPEC.md`.

STEP 2 Handoff & Implementation — OpenAI Codex

Implement the math inside `harness.py`: read all layer outputs, surface each layer's raw scalar, compute τ_{joint} , and save the fused state to the output contract in section 8.

STEP 3 Telemetry Visualization — Antigravity

Modify `build.py` and `dashboard.html` to render a live, glowing Cross-Layer Threat Alert panel that visualizes the joint threat level dynamically from the fused-state JSON.

4. Fusion Architecture (Step 1 Design)

4.1 Inputs: the raw per-layer metric

Fusion needs a continuous scalar per layer, not the pass/fail summary the harness writes today. Each driver must surface the scalar below.

Layer	Driver	Raw scalar	Symbol	Direction
Perception	drivers/perception.py	peak window gradient-energy ratio	r_win	high = anomalous
Navigation	drivers/navigation.py	peak Normalized Innovation Squared	d_nis	high = anomalous
Communication	drivers/communication.py	occupancy power in band	P	high = anomalous
CAN (software)	drivers/invehicle_can.py	observed bus message rate	rho	high = anomalous
CAN (FPGA/RTL)	drivers/invehicle_fpga.py	min inter-injection cycle gap	c	low = anomalous

4.2 Per-layer normalization

Every layer is reduced to a bounded anomaly value ϕ_i in (0,1) via a signed dimensionless exceedance, then a logistic squash.

Log-ratio (high): $e_i = (1/a_i) \cdot \ln(m_i / \tau_i)$
Log-ratio (low): $e_i = (1/a_i) \cdot \ln(\tau_i / m_i)$
Linear (NIS): $e_i = (m_i - \tau_i) / a_i$
Squash: $\phi_i = \sigma(e_i) = 1 / (1 + e^{-e_i})$

By construction $\phi_i = 0.5$ exactly at the layer's own hard threshold, rises toward 1 above it, and decays toward 0 in the safe region. That common anchor is what makes the layers comparable and the weights meaningful. Clamp e_i to $[-6, 6]$ so one saturated layer cannot dominate.

4.3 Calibration from clean data

Do not hand-pick the scale. Calibrate each a_i so the clean, no-attack median maps to a low common floor $\phi_i \approx 0.10$ (that is $e_i \approx -2.2$):

$$a_i = |e_i^{\text{raw}}(\text{clean median})| / z_0, \quad z_0 = 2.2$$

This ties the scale to real clean-run spread and keeps a uniform clean floor.

4.4 Default FUSION_CONFIG

Layer	type	direction	τ_i	a_i	w_i
Perception	logratio	high	4.0	0.42	0.20
Navigation	linear	high	9.21	4.60	0.20
Communication	logratio	high	τ_{comm}	0.50	0.20
CAN (software)	logratio	high	$\rho_0 \times 1.25$	0.30	0.20
CAN (FPGA)	logratio	low	c_{min}	0.30	0.20

$\tau = 9.21$ for Navigation is the 0.99 quantile of χ^2 with 2 degrees of freedom, the existing NIS gate. Weights are equal by default and must sum to 1. Safety-weighted alternative (raising Navigation and CAN): [0.15, 0.25, 0.15, 0.25, 0.20].

4.5 Joint Threat Score

$$T_{\text{joint}} = \sum_i w_i \cdot \phi_i, \quad \sum_i w_i = 1, \quad T_{\text{joint}} \in (0, 1)$$

A convex blend, comparable across runs. Each layer's contribution $w_i \cdot \phi_i$ is reported for the stacked panel breakdown.

4.6 Global alarm logic

Two independent triggers combined with OR:

- **Rule A (single-layer trip):** $\max_i \phi_i \geq 0.5$. Some layer crossed its own threshold. Reproduces today's per-layer alarms.
- **Rule B (coordinated):** $N_{\text{minor}} \geq k_{\text{min}}$ AND $T_{\text{joint}} \geq T^*$, where $N_{\text{minor}} = \#\{i : \phi_i \geq \phi_{\text{minor}}\}$. The new capability: fires on simultaneous minor anomalies with no single threshold crossed.

Defaults: $\phi_{\text{minor}} = 0.30$, $k_{\text{min}} = 2$, $T^* = 0.22$.

Threat level	Condition
NOMINAL	$T_{\text{joint}} < 0.15$
ELEVATED	$0.15 \leq T_{\text{joint}} < 0.22$
ALERT	$T_{\text{joint}} \geq 0.22$ or Rule A fires
CRITICAL	$T_{\text{joint}} \geq 0.45$ or ≥ 2 hard trips

4.7 Why these numbers (worked cases, equal weights, clean floor $\phi \approx 0.10$)

Scenario	T_{joint}	Outcome
All clean	≈ 0.10	No alarm
One layer at its threshold ($\phi=0.5$), rest clean	0.18	Rule A only; Rule B does not fire ($0.18 < 0.22$)
Two minor layers at $\phi=0.40$, rest clean	0.22	Rule B fires (coordinated)
Three minor layers at $\phi=0.40$	≈ 0.28	ALERT

$T^* = 0.22$ sits above the lone-layer case (0.18) and at the two-minor-layer case (0.22), which is exactly the separation intended: a single layer cannot masquerade as a cross-layer event, but two coordinated minor layers do.

4.8 Robustness

- Missing or failed layer: drop it and renormalize the remaining weights to sum to 1.
- Fewer than $k_{\min}$ layers available: disable Rule B, fall back to Rule A.
- Clamp e_i to $[-6, 6]$; floor log inputs at a small epsilon.

5. Capabilities Delivered

- **Coordinated sub-threshold detection.** Raises a global alarm on several simultaneous minor anomalies that every individual detector would miss.
- **Comparable layers.** A single logistic anchor ($\phi = 0.5$ at each layer's own threshold) puts five very different metrics onto one scale.
- **One interpretable score.** T_{joint} in $(0,1)$ with per-layer contributions, ready to drive a single meter and a stacked breakdown.
- **Self-calibrating.** Scales derive from clean-run medians, not guesswork.
- **Honest false-alarm control.** Validation gates the fused false-alarm rate so fusion does not cost the project its clean record.
- **Graceful degradation.** Skipped or failed layers are excluded and weights renormalize automatically.

6. Handoffs

6.1 To OpenAI Codex (Step 2)

- In each driver, add two keys to the JSON record it already prints: "fusion_metric": "r_win", "fusion_value": 3.4.
- Source of each scalar: Perception = max window gradient-energy ratio on the tested frame; Navigation = peak NIS over the post-event window; Communication = occupancy power in the monitored band; CAN software = observed message rate; FPGA = minimum inter-injection cycle gap in the testbench trace.
- In harness.py, add FUSION_CONFIG and a fuse(records) step after the layer loop; write the section 8 object. Keep all constants in FUSION_CONFIG.

6.2 To Antigravity (Step 3)

- Render joint_threat_score as the glowing meter.
- Use level for its color (NOMINAL / ELEVATED / ALERT / CRITICAL).
- Render per-layer contribution as a stacked breakdown.
- Drive the alert text from hard_trips and alarm_reason.

7. Output Contract (fused state)

harness.py writes this object (sibling fusion.json or a top-level key). It is the interface Step 3 consumes.

```
{
  "joint_threat_score": 0.27,
  "level": "ALERT",
  "global_alarm": true,
  "alarm_reason": "coordinated",      // "single-layer" | "coordinated" | null
  "n_minor": 3,
  "hard_trips": [],
  "weights_mode": "equal",
  "config": { "T_low": 0.15, "T_star": 0.22, "T_crit": 0.45,
              "phi_minor": 0.30, "k_min": 2, "z0": 2.2 },
  "layers": [
    { "layer": "Perception (camera / VLM input)", "metric": "r_win",
      "value": 3.4, "threshold": 4.0, "exceedance": -0.39, "phi": 0.40,
      "weight": 0.20, "contribution": 0.080, "minor": true, "trip": false }
  ]
}
```

8. Reference Implementation (drop-in for the harness)

```
import math

# type, direction, tau, a, w (tau may be resolved at runtime from the driver)
FUSION_CONFIG = {
    "adversarial-patch-detector": ("logratio", "high", 4.0, 0.42, 0.20),
    "ekf-gps-spoof-detector": ("linear", "high", 9.21, 4.60, 0.20),
    "v2x-jamming-detector": ("logratio", "high", None, 0.50, 0.20),
    "canbus-ids": ("logratio", "high", None, 0.30, 0.20),
    "canbus-ids-fpga": ("logratio", "low", None, 0.30, 0.20),
}

T_LOW, T_STAR, T_CRIT = 0.15, 0.22, 0.45
PHI_MINOR, K_MIN, EPS = 0.30, 2, 1e-9

def _exceedance(kind, direction, value, tau, a):
    value, tau = max(value, EPS), max(tau, EPS)
    if kind == "linear":
        e = (value - tau) / a
    else:
        r = math.log(value / tau)
        e = r / a if direction == "high" else -r / a
    return max(-6.0, min(6.0, e))

def _phi(e):
    return 1.0 / (1.0 + math.exp(-e))

def fuse(records):
    layers, wsum = [], 0.0
    for r in records:
        cfg = FUSION_CONFIG.get(r.get("repo"))
        if not cfg or not r.get("ok") or r.get("fusion_value") is None:
            continue
        kind, direction, tau_default, a, w = cfg
        tau = r.get("fusion_threshold", tau_default)
        if tau is None:
            continue
        e = _exceedance(kind, direction, float(r["fusion_value"]), float(tau), a)
        phi = _phi(e)
        layers.append({
            "layer": r.get("layer"), "metric": r.get("fusion_metric"),
            "value": r["fusion_value"], "threshold": tau,
            "exceedance": round(e, 3), "phi": round(phi, 3), "weight": w,
            "trip": phi >= 0.5, "minor": PHI_MINOR <= phi < 0.5,
        })
        wsum += w
    if not layers:
        return {"joint_threat_score": 0.0, "level": "NOMINAL",
                "global_alarm": False, "alarm_reason": None, "layers": []}
    t_joint = 0.0
    for L in layers:
        L["weight"] = L["weight"] / wsum
        L["contribution"] = round(L["weight"] * L["phi"], 3)
        t_joint += L["weight"] * L["phi"]
    n_minor = sum(1 for L in layers if L["phi"] >= PHI_MINOR)
    hard = [L["layer"] for L in layers if L["trip"]]
    rule_a = len(hard) >= 1
```

```

rule_b = (len(layers) >= K_MIN) and (n_minor >= K_MIN) and (t_joint >= T_STAR)
if t_joint >= T_CRIT or len(hard) >= 2:
    level = "CRITICAL"
elif t_joint >= T_STAR or rule_a:
    level = "ALERT"
elif t_joint >= T_LOW:
    level = "ELEVATED"
else:
    level = "NOMINAL"
reason = "single-layer" if rule_a else ("coordinated" if rule_b else None)
return {"joint_threat_score": round(t_joint, 3), "level": level,
        "global_alarm": rule_a or rule_b, "alarm_reason": reason,
        "n_minor": n_minor, "hard_trips": hard, "weights_mode": "equal",
        "config": {"T_low": T_LOW, "T_star": T_STAR, "T_crit": T_CRIT,
                    "phi_minor": PHI_MINOR, "k_min": K_MIN, "z0": 2.2},
        "layers": layers}

```

9. Validation Requirements

Fusion must not cost the project its clean record. Validate on clean, no-attack runs and confirm the fused false-alarm rate is at or below target (aim 0.00, accept at most 0.01). If clean runs push T_{joint} near T^* , raise T^* or recalibrate the scales (section 4.3) until the clean floor sits near 0.10. Then confirm a synthetic coordinated case, two or more layers held in the minor band with none crossing its own threshold, produces a global alarm with `alarm_reason = "coordinated"`.

10. Actions Taken (Claude Code, Step 1)

#	Action	Result
1	Inspected results.json	Confirmed the harness records only per-layer pass/fail summaries, not the continuous scalars fusion needs.
2	Read harness.py	Mapped the subprocess-driver architecture and the JSON record each driver emits.
3	Designed the normalization	Signed exceedance plus logistic squash, anchored at $\phi = 0.5$ at each layer's own threshold; self-calibrating scales.
4	Designed the Joint Threat Score and alarm logic	Convex-weighted T_{joint} ; OR of single-layer Rule A and coordinated Rule B, with worked thresholds.
5	Defined the output contract and handoffs	Exact JSON schema and driver keys for Codex and the dashboard.
6	Authored the spec	Wrote FUSION_SPEC.md to the project root, then removed a stray code fence.

Deliverables

- `av-stack-defense/FUSION_SPEC.md` — the authoritative Step 1 specification.
- `av-stack-defense/docs/Cross-Layer-Signal-Fusion-Report.pdf` — this report.

Open item for the group

Three thresholds are resolved at runtime from the drivers rather than hard-coded: `tau_comm` (communication occupancy-power threshold), `rho_0` (normal CAN rate), and `c_min` (FPGA minimum spacing). Codex should read these from the respective drivers during implementation, or request that Claude Code pin default values.

Cross-Layer Signal Fusion — Multi-Agent Collaboration Report · av-stack-defense. Design authored by Claude Code (Step 1). Implementation and visualization proceed against the contract in sections 7 and 8.

11. Implemented Extensions and Validated Results

The following are implemented end to end and measured from a real harness run, not asserted. They fold in the three requested directions: self-calibration, safety-critical weights, and the coordinated cross-layer attack proof.

11.1 Self-calibration from clean data

Each driver runs its clean scenario over many seeds and emits a distribution of clean peaks. The harness sets each scale so the clean median maps to $\phi_i = 0.10$. No hand-tuned constants remain; the harness self-calibrates to any baseline.

11.2 Coordination eligibility (false-alarm guard)

A layer may drive the global fused alarm only if its clean 95th-percentile ϕ_i stays below ϕ_{i_minor} . An ineligible layer is excluded from the score and both rules; its own detector stays authoritative. Measured:

Layer	clean ϕ_i p95	eligible
Perception (peak window ratio)	0.327	no
Navigation	0.100	yes
Communication	0.100	yes
CAN (software)	0.101	yes
CAN (FPGA)	0.100	yes

11.3 Weight profiles

equal (0.20 each) and **safety-critical** (Navigation and CAN raised to 0.25, Perception and Communication 0.15), both written by the harness.

11.4 Coordinated cross-layer attack, proven

Demonstrated through the real detectors. Navigation carries a brief GPS spike tuned to peak NIS 8.997 (under the 9.21 gate); Communication carries a weak tone tuned just under its power gate.

Layer	phi	state	local detector
Navigation	0.414	minor	silent
Communication	0.355	minor	silent
CAN, FPGA	0.10	clean	silent
Perception	0.10	excluded	silent

Result. Every local detector silent, no single layer tripped, yet $\tau_{\text{joint}} = 0.242$, the fused alarm fires with `alarm_reason = "coordinated"` and `thesis_proven = true`. The cross-layer thesis, made concrete.

11.5 Held-out false-alarm validation

Clean peaks are split 50/50; the scale is calibrated on one split and the fused alarm is Monte-Carlo evaluated on the independent held-out split (8000 trials). Because calibration and evaluation use disjoint clean data, this is a real generalization estimate.

Measured fused false-alarm rate: 0.0, with clean τ_{joint} 95th percentile at 0.100.

11.6 Output files

`results.json`, `fusion.json` / `fusion_safety.json`, `fusion_clean.json` / `fusion_clean_safety.json`, `fusion_coordinated.json` / `fusion_coordinated_safety.json`, and `fusion_validation.json`. New fields include `n_minor_eligible`, `hard_trips_eligible`, `coordination_eligible`, `in_score`, `individual_safe`, `all_individual_safe`, and `thesis_proven`.